3. Arrays

Sitio:	Centros Granada 26	Imprimido por:	Triay Orfila, Marc
Curso:	Desarrollo web en entorno servidor	Día:	lunes, 5 de octubre de 2026, 17:13
Libro:	3. Arrays		

Tabla de contenidos

1. Creando arrays

2. Arrays multidimensionales

3. Funciones de arrays

4. Ejemplos de funciones

5. isset() y empty()

1. Creando arrays

Un array es una colección de valores con un único nombre.

Como ya vimos anteriormente, los arrays en PHP son una estructura muy flexible y potente. Unifica en un solo tipo lo que en otros lenguajes se consigue con arrays básicos, vectores, lista o diccionarios.

Para acceder a los distintos valores de la variable se utiliza un índice numérico o alfanumérico.

Existen varias **formas de crear un array** en PHP:

- La forma más simple de crear un array es mediante asignación directa; ya que la primera vez que asignemos un valor, el array se creará en el entorno.

```php
<?php
$miArray[] = 45; // Empieza con índice 0
$miArray[] = 2; // Índice 1
$miArray[2] = 5; // Asignación directa a un índice en concreto
echo " Este es el primer array que hemos creado <br>";
print_r($miArray);
```

Se puede acceder a cada elemento del array por su clave utilizando los corchetes: $arr[clave] = valor;

- Con la función array() .

```
$var = array (
clave1 => valor1,
…..
claven => valorn,
);
```

También podemos usar la notación entre corchetes:

```
$arr = array [
clave1 => valor1,
…..
claven => valorn,
];
```

Los índices se añaden automáticamente empezando desde 0. Si no se asignan parámetros la función devolverá un array vacío.

```
<?php
$otroArray = array ( 34,55,77); // usando la función array
echo "<br> <br>Este array se ha creado con la función array:
<br>";
print_r($otroArray);
```

Podemos añadir índices numéricos o alfanuméricos en la llamada a la función array() e incluso mezclar los tipos:

```
$otroArray = array ( "nombre"=> "Maria", "edad"=>22,
3=>"soltera");
echo "<br> <br>Ejemplo con índices alfanuméricos: <br>";
print_r($otroArray);
```

Se pueden usar igualmente los corchetes para hacer la asignación en lugar de la función array(), lo que hace que el código sea más fácil de leer.

```
$miOtroArray = [33,44,55,66]; // usando corchetes es más limpio
echo "<br> <br>Este array se ha creado usando corchetes: <br>";
print_r($miOtroArray);
```

Y de la misma forma también podemos poner con corchetes los índices alfanuméricos:

```
$miOtroArray = ["nombre"=> "Maria", "edad"=>22, 3=>"soltera"];
echo "<br> <br>El resultado es el mismo con corchetes: <br>";
print_r($miOtroArray);
```

- Utilizando funciones que devuelven arrays. Existen funciones que devuelven este tipo de datos. Por ejemplo: range().

```
<?php
$mis_numeros = range(5,20);
echo "<pre>";
var_dump($mis_numeros);
```

Para **recuperar** cualquier valor se usa el índice dentro de los corchetes:

Ejemplo `echo $miOtroArray["nombre"];`

También puedo recuperar los valores usando la **función list()** que los almacenará en una lista de variables. El número de variables que contienen la función debe ser igual al número de datos almacenados en el array.

```php
<?php
$mis_numeros = array (12,44,35,66);
list($a,$b,$c,$d) = $mis_numeros;
echo "los números que tengo en el array son:  $a, $b, $c, $d";
```

Para **recorrer un array** lo habitual es utilizar el bucle foreach.

```
foreach($arr as $valor){
….
}
```

El cuerpo del bucle se ejecuta una vez por cada elemento del array $arr. En cada iteración del bucle $valor almacena elemento correspondiente.

También se puede utilizar especificando una variable para la clave, **foreach($arr as $clave =>$valor){…}**

En este caso, en cada iteración también se dispondrá de la clave del elemento correspondiente en la variable $clave.

Ejemplo:

```php
<?php

$alumnos = [
    [
        "nombre" => "Ana",
        "apellidos" => "López"
    ],
    [
        "nombre" => "Luis",
        "apellidos" => "García"
    ],
    [
        "nombre" => "María",
        "apellidos" => "Martínez"
    ]
];

foreach ($alumnos as $alumno) {

    foreach ($alumno as $clave => $valor) {
        echo $clave . ": " . $valor . " ";
    }

    echo "<br>";
}
```

2. Arrays multidimensionales

En PHP puedes crear también arrays de varias dimensiones almacenando otro array en cada uno de los elementos de un array.

El siguiente ejemplo muestra como almacenar los contactos de una agenda:

```
$contactos = [
    [
        'nombre' => 'Ana',
        'apellidos' => 'García',
        'telefono' => '677564732'
    ],
    [
        'nombre' => 'Andres',
        'apellidos' => 'González',
        'telefono' => '677564442'
    ],
    [
        'nombre' => 'Marta',
        'apellidos' => 'Pérez',
        'telefono' => '777564731'
    ]
];
```

También podemos expresarlo en esta otra nomenclatura:

```
<?php
$contactos = array(
    array(  // aqui tengo todos los indices asociativos de mis contactos
        'nombre' =>'Ana',
        'apellidos' => 'García',
        'telefono' => '677564732'
    ),
    array(
        'nombre' =>'Andres',
        'apellidos' => 'González',
        'telefono' => '677564442'
    ),
    array(
        'nombre' =>'Marta',
        'apellidos' => 'Pérez',
        'telefono' => '777564731'
    )

);
```

Para hacer referencia a los elementos almacenados en un array multidimensional, debes indicar las claves para cada una de las dimensiones:

$contactos[0]['nombre']

Otro ejemplo:

Supongamos que queremos crear una matriz bidimensional con las calificaciones obtenidas por una alumna en las tres evaluaciones de Matemáticas, Lengua y Dibujo.

Podríamos hacerlo así:

```
$notas["Mat"]["Pri"] = "Suficiente";
$notas["Mat"]["Seg"] = "Bien";
$notas["Mat"]["Ter"] = "Notable";
$notas["Len"]["Pri"] = "Sobresaliente";
$notas["Len"]["Seg"] = "Sobresaliente";
$notas["Len"]["Ter"] = "Notable";
$notas["Dib"]["Pri"] = "Notable";
$notas["Dib"]["Seg"] = "Bien";
$notas["Dib"]["Ter"] = "Suficiente";
```

O bien, así:

```
$notas = array("Mat"=>array("Pri"=>"Suficiente",
                            "Seg"=>"Bien",
                            "Ter"=>"Notable"),
               "Len"=>array("Pri"=>"Sobresaliente",
                            "Seg"=>" Sobresaliente",
                            "Ter"=>"Notable"),
               "Dib"=>array("Pri"=>"Notable",
                            "Seg"=>"Bien",
                            "Ter"=>"Suficiente"));
```

Por supuesto también podíamos haber usado índices numéricos para las asignaturas y las calificaciones:

```
$notas = array(0=>array(0=>5,1=>6,2=>7),
               1=>array(0=>9,1=>9,2=>7),
               2=>array(0=>7,1=>6,2=>5));
```

Aunque de esta forma la declaración es más breve, sin embargo, en el futuro habrá que recordar a qué asignatura y a que evaluación corresponde cada índice

3. Funciones de arrays

Consultaremos las funciones de arrays en el manual de PHP.

Algunas interesantes son:

- count() Número de elementos del array
- in_array($valor, $arr) Busca un valor en el array
- unset() Esta función es la misma con la que se borran las variables definidas
- sort(). Ordenación ascendente. Se pierde el índice asociativo
- ksort() . Ordenación ascendente de las parejas índice/valor atendiendo al índice.
- asort(). Ordenación ascendente de las parejas índice/valor atendiendo al dato. Buen método para arrays asociativos
- arsort() Igual a asort() pero descendente
- rsort() . Ordenación descendente
- krsort(). Igual a Ksort pero descendente
- array_values() Todos los valores del array
- array_keys() . Devuelve las claves del array
- array_key_exits($arr,$cla) Verifica si el índice o la clave existen en el array
- array_reverse() Orderar a la inversa
- array_push(). Inserta un valor . Pilas LIFO
- array_pop() Extraer un valor. El último de la pila LIFO
- array_flip() Intercambia los valores de índices y datos
- current() Posición actual del puntero en el array
- next(). Avanza a la siguiente posición
- prev() Retrocede una posición

4. Ejemplos de funciones

- Comprobar si un valor está en un array con in_array:

```php
$carrito = ['Tablet', 'Portatil', 'Television'];

var_dump( in_array('Tablet', $carrito) );
var_dump( in_array('Audifonos', $carrito) );
```

- Ordenar elementos de un array:

```php
$numeros = array(1,3,4,5,1,2);
sort($numeros); // de menor a mayor
print_r($numeros);

rsort($numeros); // de mayor a menor

echo "<pre>";
var_dump($numeros);
echo "</pre>";
```

- Ordenar un array asociativo:

Partimos del siguiente array

```php
$cliente = array(
    'saldo' => 200,
    'tipo' => 'Premium',
    'nombre' => 'Juan'
);
```

  - Ordenando los valores

```php
asort($cliente); // Ordena por valores (orden alfabetico)
print_r($cliente);

arsort($cliente); // Ordena por valores (Z primero)
print_r($cliente);
```

  - Ordenando las claves

```
ksort($cliente); // ordena por keys (orden alfabetico);
krsort($cliente); // ordena por keys (orden alfabetico, DE LA Z a la A);
```

- Usando las funciones prev() y next(). En el siguiente ejemplo se recorre un array:

```
$ciudades = array("Badajoz", "Mérida", "Cáceres", "Plasencia", "Trujillo");
$ciudades["España"] = "Madrid";
$ciudades["Portugal"] = "Lisboa";
$ciudades["Francia"] = "Paris";

function recorre($ciudades) {
    do {
        $ciudad_actual = current($ciudades);
        echo "El valor es: $ciudad_actual <br/>";
    } while (next($ciudades));
}

function recorre_inverso($ciudades) {
    end($ciudades);
    do {
        $ciudad_actual = current($ciudades);
        echo "El valor es: $ciudad_actual <br/>";
    } while (prev($ciudades));
}

recorre($ciudades);
echo "<hr/>";
recorre inverso($ciudades);
```

- Intercambiar todas las claves de un array con sus valores asociados usando array_flip()

```php
function recorre($numeros) {
    foreach($numeros as $indice => $valor) {
        echo "$indice: $valor <br/>";
    }
}

$numeros = array("uno" => 1, "dos" => 2, "tres" => 3, "cuatro" => 4);
echo "Números <br/>";
echo "<hr/>";
recorre($numeros);
echo "Números intercambiados <br/>";
echo "<hr/>";
recorre(array_flip($numeros));
```

- Ejemplo de uso de implode() y explode():

```php
// explode()

$texto="blanco;amarillo;rojo;verde;azul;negro";
$colores=explode(";",$texto);
echo "<h1>Función explode()</h1><P>
        <h3>
            Hemos creado la variable \$texto=\"$texto\"<P>
            En el array \$colores creada con la función explode()
            tenemos los elementos siguientes:
        </h3>
        <p>";

$numero_elementos=count($colores);
for ($i=0; $i < $numero_elementos; $i++)
    echo "Elemento $i:<B> $colores[$i]. </B></P>";

// implode()

echo "<h1>Función implode()</h1><P>
        <h3>Ahora convertimos en la cadena <B>\$texto </B>
            los elementos de la matriz <b>\$colores</b> usando la
            funcion <b>implode(\$colores)</b>.
        </h3>
    </P>";
$texto=implode(";",$colores);
echo "La variable <b>\$texto</b> contiene de nuevo <b>\"$texto\"</b>.";
```

5. isset() y empty()

Es importante saber diferenciar entre si un array está vacío o si está creado.

- Para saber si un array está vacío usaremos la función **empty().**

Usa el siguiente ejemplo para comprobarlo:

```
$clientes = [];
$clientes2 = array();
$clientes3 = array('Pedro', 'Juan', 'Karen');


// Empty - Revisa si un array esta vacio
var_dump( empty($clientes) );
var_dump( empty($clientes3) );
var_dump( empty($clientes2) );
```

- Para revisar si un array esta creado o una propiedad esta definida se usa la función isset()

A partir de los valores anteriores comprueba los resultados de ejecutar el siguiente código

```
var_dump( isset($clientes4) );
var_dump( isset($clientes) );
var_dump( isset($clientes2) );
var_dump( isset($clientes3) );
```

- La función isset() también se puede usar en los arrays asociativos para saber si una key existe dentro de ese array. Por ejemplo, vamos a definir un nuevo array asociativo con los siguientes valores:

```
$cliente = [
    'nombre' => 'Juan',
    'saldo' => 200
];
```

Y usaremos isset() para comprobar si existen las siguientes claves en el array:

```
var_dump( isset($cliente['nombre']) );
var_dump( isset($cliente['codigo']) );
```

Reiniciar tour para usuario en esta página